

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

PROYECTO FIN DE CURSO (PFC) — ENTREGA 4

PROYECTO:

**TIENDATECH: CALIDAD, OBSERVABILIDAD Y
EVALUACIÓN EXPERIMENTAL DEL SISTEMA
DISTRIBUIDO**

MATERIA:

APLICACIONES DISTRIBUIDAS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CURSO / PERÍODO ACADÉMICO:

7mo A — PERÍODO ACADÉMICO 2026

INTEGRANTES:

Jhinson Stalyn Aucatoma Celorio
Jeremy Ruperto Gaibor Rodriguez
Andy Paul Sanchez Pilaloa
José Alejandro Lozano Morales

FECHA DE ENTREGA:

Martes 25 de agosto del 2026

Índice

Resumen	3
Abstract	4
1. Introducción	5
1.1. Alcance y contribución delimitada de la Entrega 4	5
2. Trabajos relacionados	6
2.1. Bases de datos distribuidas	6
2.2. Arquitectura de microservicios	6
2.3. Ingeniería de aplicaciones móviles	6
2.4. Calidad de software y CI/CD	7
2.5. Matriz comparativa	7
3. Fundamento teórico	8
3.1. Arquitectura en capas y principios SOLID	8
3.2. Patrones de diseño GoF	8
3.3. Ingeniería de aplicaciones móviles	9
3.4. Integración y entrega continuas (CI/CD)	9
3.5. Observabilidad distribuida	9
3.6. Modelo de calidad ISO/IEC 25010:2023	9
4. Arquitectura del sistema	10
4.1. Vistas C4 y evolución respecto a E3	10
4.2. Refactor en capas y principios SOLID (D1.1)	11
4.3. Patrones GoF aplicados (D1.2)	12
4.4. Persistencia distribuida operando (D4.2)	13
4.5. Stack tecnológico	14

5. Aplicación web	15
5.1. Funcionalidad (D2.1)	15
5.2. Calidad (D2.2)	16
6. Aplicación móvil	17
6.1. Funcionalidad (D3.1)	17
6.2. Calidad (D3.2)	18
7. Pruebas y CI/CD	19
7.1. Pirámide de pruebas (D5.1)	19
7.2. Contratos de consumidor Pact (D4.1)	19
7.3. Pipeline CI/CD end-to-end (D5.2)	20
8. Observabilidad	20
9. Evaluación ISO/IEC 25010	21
10. Discusión y amenazas a la validez	21
10.1. Interpretación de resultados y alcance no cubierto	21
10.2. Amenazas a la validez (D9.3)	22
10.3. Reflexión ética (D9.2)	22
10.4. Reproducibilidad (D9.4)	23
11. Conclusiones y trabajo futuro	24
11.1. Cierre del ciclo E1–E4	24
11.2. Conclusiones	25
11.3. Trabajo futuro	26

Resumen

Objetivo: cerrar, sobre el sistema distribuido **TiendaTech** validado en la Entrega 3, la brecha entre un sistema que funciona y uno que puede sostenerse, verificarse y evolucionar de forma controlada, mediante refactor arquitectónico, persistencia distribuida segura, calidad de las aplicaciones cliente y automatización del ciclo de integración continua. **Método:** auditoría directa del código fuente frente a cada criterio de la rúbrica de la Entrega 4, declarando explícitamente qué se cumple, qué se cumple de forma parcial y qué queda fuera de alcance, en lugar de reportar cobertura no verificada. **Resultados:** los seis microservicios Java quedaron organizados en cuatro capas con puertos y adaptadores, con cinco patrones GoF documentados con evidencia real; la persistencia distribuida en CockroachDB migró a una conexión cifrada por certificado (mTLS) con *healthchecks* en los siete servicios; la aplicación web expone doce rutas y la aplicación móvil usa dos capacidades del dispositivo con pruebas unitarias e instrumentadas; el pipeline de CI/CD cubre de forma parcial cuatro de los siete *jobs* esperados; y las pruebas del backend, los contratos Pact, la observabilidad distribuida y la evaluación ISO/IEC 25010 quedan con cobertura desigual o explícitamente pendientes. **Conclusiones:** el sistema avanzó de forma genuina pero incompleta hacia la sostenibilidad operativa, documentada con honestidad de alcance en cada sección.

Palabras clave: Microservicios, Arquitectura en capas, Persistencia distribuida, CockroachDB, Calidad de software, CI/CD.

Abstract

Objective: to close, over the **TiendaTech** distributed system validated in Deliverable 3, the gap between a system that works and one that can be sustained, verified, and evolved in a controlled way, through architectural refactoring, secure distributed persistence, client application quality, and continuous integration automation. **Method:** a direct source-code audit against each criterion of the Deliverable 4 rubric, explicitly stating what is fully met, partially met, or out of scope, instead of reporting unverified coverage. **Results:** the six Java microservices were reorganized into four layers with ports and adapters, with five GoF design patterns documented with real code evidence; distributed persistence on CockroachDB migrated to a certificate-based encrypted connection (mTLS) with healthchecks on all seven services; the web application exposes twelve routes and the mobile application uses two device capabilities with unit and instrumented tests; the CI/CD pipeline partially covers four of the seven expected jobs; and backend testing, Pact contracts, distributed observability, and the ISO/IEC 25010 evaluation remain unevenly covered or explicitly pending. **Conclusions:** the system advanced genuinely, though incompletely, toward operational sustainability, documented with honest scope declarations throughout.

Keywords: Microservices, Layered architecture, Distributed persistence, CockroachDB, Software quality, CI/CD.

1. Introducción

La Entrega 3 de **TiendaTech** validó un despliegue contenerizado compuesto por un frontend y seis microservicios conectados a una base de datos distribuida CockroachDB, confirmando el flujo transaccional básico de carrito, orden, factura e inventario sobre esa infraestructura. Esa entrega dejó explícitamente fuera de su contribución delimitada el aislamiento de persistencia por servicio, la alta disponibilidad productiva, el motor de compatibilidad de componentes y el asistente conversacional, así como la calidad de código, la observabilidad y la evaluación experimental formal del sistema.

La Entrega 4 retoma el sistema en ese punto y se concentra en cerrar la brecha entre *tener un sistema distribuido que funciona* y *tener un sistema distribuido que puede sostenerse, verificarse y evolucionar de forma controlada*. En este período se completó la migración de los microservicios afectados hacia una organización en capas explícita (**presentation/application/domain/infrastructure**), se documentaron los patrones de diseño GoF aplicados, se aseguró la persistencia distribuida con una conexión cifrada por certificados (mTLS) hacia el clúster CockroachDB en lugar de la conexión directa sin cifrar heredada de E3, y se avanzó en la calidad de las aplicaciones cliente (web y móvil) y en la automatización del ciclo de integración continua.

1.1. Alcance y contribución delimitada de la Entrega 4

Siguiendo el mismo principio de honestidad de alcance aplicado en las entregas anteriores, esta sección declara explícitamente qué criterios de la Guía Integral de la Entrega 4 cubre este documento y cuáles quedan fuera.

Se abordan de forma directa: el refactor en capas y los patrones GoF con su respectivo registro de decisión arquitectónica (D1); el estado de las aplicaciones web y móvil, tanto en funcionalidad como en calidad de código (D2, D3); la persistencia distribuida operando de forma segura y, en la medida en que el tiempo del equipo lo permitió, los contratos de consumidor entre los clientes y el backend (D4); la pirámide de pruebas y el pipeline de integración continua (D5); la trazabilidad de la evolución del proyecto entre E1 y E4 (D8.3); y la reflexión ética, las amenazas a la validez y la reproducibilidad del entorno de despliegue (D9).

Quedan explícitamente fuera del alcance de esta entrega la instrumentación de observabilidad distribuida (OpenTelemetry, Prometheus, Grafana) y, como consecuencia directa de ello, la evaluación experimental basada en ISO/IEC 25010 (D6 y D7), dado que esta última depende de contar con las métricas operativas que solo la primera puede producir. Las Secciones 8, 9 y 10 retoman esta decisión de alcance con más detalle. Esta exclusión se declara aquí de forma explícita, antes de que el lector avance en el documento, para no generar la expectativa de una cobertura que el equipo no pudo verificar en el tiempo disponible.

2. Trabajos relacionados

Esta sección revisa siete fuentes indexadas — dos ya citadas en `docs/entrega3/PFC3.tex` y reutilizadas aquí por seguir siendo pertinentes, y cinco incorporadas específicamente para esta entrega — organizadas en los cuatro ejes que exige la rúbrica de la Entrega 4: bases de datos distribuidas, arquitectura de microservicios, ingeniería de aplicaciones móviles y calidad de software/CI-CD.

2.1. Bases de datos distribuidas

Taft, Sharif, Matei et al. [1] describen la arquitectura de CockroachDB — el motor efectivamente usado por TiendaTech desde E3 — detallando cómo fragmenta las tablas en rangos replicados por consenso Raft y las lecciones aprendidas al adaptar Raft a un sistema de producción (agrupar *heartbeats*, pausar grupos Raft inactivos, reconfiguración por consenso conjunto). Özsü y Valduriez [2] complementan esto con el marco formal de fragmentación horizontal primaria y derivada que TiendaTech ya usó para justificar el particionamiento de `pedidos.orden` por trimestre en E3 (ADR-003). Ninguna de las dos fuentes evalúa, sin embargo, el costo operativo de migrar una conexión insegura a una conexión mTLS con certificados cliente sobre un clúster ya desplegado — que es exactamente el cambio de infraestructura documentado en la Sección 4.4 de esta entrega.

2.2. Arquitectura de microservicios

Márquez, Osses y Astudillo [3] revisan sistemáticamente la literatura académica e industrial sobre patrones y tácticas arquitectónicas para microservicios, identificando 44 patrones en fuentes académicas y 74 en fuentes industriales, la mayoría asociados a escalabilidad, flexibilidad, capacidad de prueba, desempeño y elasticidad. Zhong, Li, Huang et al. [4] investigan empíricamente — combinando una revisión sistemática de 34 estudios con una encuesta a 63 profesionales — el uso de *Domain-Driven Design* para identificar los límites (*bounded contexts*) de cada microservicio, un problema de diseño previo al que TiendaTech resolvió de forma pragmática y no formalizada al dividir el dominio en usuarios, productos, inventario, pedidos, órdenes a proveedores y ventas. Ninguna de las dos fuentes profundiza en el refactor interno de un microservicio ya existente hacia una arquitectura de puertos y adaptadores en cuatro capas — que es precisamente lo que la Sección 4 documenta para los seis servicios Java de TiendaTech.

2.3. Ingeniería de aplicaciones móviles

Nawrocki, Wrona, Marczak et al. [5] comparan empíricamente aplicaciones idénticas desarrolladas con frameworks nativos y multiplataforma (Kotlin nativo frente a React Native y Flutter,

entre otros), evaluando funcionalidad, esfuerzo de desarrollo y rendimiento; sus resultados muestran que el desarrollo nativo ofrece el mayor rendimiento y el acceso más directo a las capacidades del dispositivo, a costa de mayor esfuerzo de desarrollo por no compartir código entre plataformas. Este hallazgo es directamente relevante para TiendaTech: la aplicación móvil (Sección 6) se construyó en Kotlin nativo con Jetpack Compose precisamente para acceder sin capas intermedias a CameraX/ML Kit (escaneo de códigos de barras) y a la API de notificaciones del sistema operativo, las dos capacidades de dispositivo requeridas por la rúbrica — una elección que este trabajo relacionado ayuda a justificar cuantitativamente, aunque TiendaTech mismo no ejecutó un benchmark comparativo propio frente a una alternativa multiplataforma.

2.4. Calidad de software y CI/CD

Shahin, Babar y Zhu [6] ofrecen la revisión sistemática de referencia sobre integración, entrega y despliegue continuos, clasificando enfoques, herramientas, desafíos y prácticas de la industria; su marco conceptual de *pipeline* como *quality gate* automatizado es el que se usa para evaluar, en la Sección 7.3, qué tan completo está el pipeline real de TiendaTech frente al ideal de siete *jobs* de la rúbrica. Hui, Wang, Li et al. [7] aportan el estudio más reciente y más específico para este proyecto: un mapeo sistemático de métodos de prueba para sistemas basados en microservicios, que identifica nueve categorías de pruebas (incluidas pruebas unitarias, de contrato y extremo a extremo) y los desafíos de aplicarlas cuando los servicios se despliegan y actualizan de forma independiente — exactamente la situación de TiendaTech, donde la cobertura de pruebas es desigual entre sus seis microservicios (Sección 7.1). Ninguna de las dos fuentes mide el costo de retrasar la adopción de pruebas de contrato (Pact) cuando el tiempo del equipo es limitado, que es la decisión de priorización que TiendaTech documenta explícitamente en la Sección 7.2.

2.5. Matriz comparativa

La Tabla 1 condensa el aporte de cada fuente y la brecha que deja abierta respecto del trabajo realizado en TiendaTech.

Cuadro 1: Matriz comparativa de trabajos relacionados (Entrega 4).

Referencia	Aporte principal	Brecha frente a TiendaTech E4
Taft, Sharif, Matei et al. [1]	Arquitectura de CockroachDB y adaptación de Raft a producción.	No evalúa migrar una conexión insegura a mTLS sobre un clúster ya desplegado.
Özsu y Valduriez [2]	Marco formal de fragmentación horizontal primaria/derivada.	Supone conocida la carga de consultas desde el diseño inicial.
Márquez, Osses y Astudillo [3]	44+74 patrones/tácticas arquitectónicas para microservicios.	No cubre el refactor interno de un servicio ya existente hacia capas.
Zhong, Li, Huang et al. [4]	DDD para identificar límites de microservicios (34 estudios, 63 encuestados).	No aborda la reestructuración posterior de un microservicio ya delimitado.
Nawrocki, Wrona, Marczak et al. [5]	Comparación empírica nativo vs. multiplataforma (funcionalidad, esfuerzo, rendimiento).	TiendaTech no ejecutó un benchmark propio frente a una alternativa multiplataforma.
Shahin, Babar y Zhu [6]	Revisión sistemática de CI/CD: enfoques, herramientas, prácticas.	No mide el costo de un pipeline parcialmente implementado bajo restricción de tiempo.
Hui, Wang, Li et al. [7]	Mapeo sistemático de pruebas para sistemas basados en microservicios.	No mide el costo de retrasar la adopción de pruebas de contrato (Pact).

3. Fundamento teórico

3.1. Arquitectura en capas y principios SOLID

La arquitectura en capas separa un sistema en niveles con responsabilidades distintas y dependencias unidireccionales: presentación (adapta el mundo exterior al sistema), aplicación (orquesta casos de uso), dominio (reglas de negocio y contratos) e infraestructura (detalles técnicos concretos: bases de datos, HTTP, colas). Los principios SOLID — responsabilidad única, abierto/cerrado, sustitución de Liskov, segregación de interfaces e inversión de dependencias — son las restricciones que evitan que esas capas se acoplen incorrectamente y que las clases se conviertan en objetos que concentran demasiadas responsabilidades. El uso disciplinado de SOLID es, en particular, el prerrequisito para que los patrones de diseño GoF aporten valor real: un patrón aplicado sobre un diseño que ya viola SOLID solo añade complejidad accidental.

3.2. Patrones de diseño GoF

El catálogo de patrones de diseño de Gamma, Helm, Johnson y Vlissides (1994) — conocidos como el "Gang of Four" (GoF) — documenta veintitrés soluciones recurrentes a problemas de

diseño orientado a objetos, agrupadas en patrones creacionales, estructurales y de comportamiento. Su valor no está en aplicarlos por cumplir una cuota, sino en que cada patrón resuelve un problema de acoplamiento o variabilidad concreto; por eso la Sección 4 documenta, para cada patrón aplicado en TiendaTech, el problema real que resuelve en el proyecto y no solo su definición canónica.

3.3. Ingeniería de aplicaciones móviles

El desarrollo móvil enfrenta retos que no existen (o existen de forma distinta) en el desarrollo web: heterogeneidad de dispositivos y versiones de sistema operativo, restricciones de batería y de red intermitente, un ciclo de vida de la aplicación gestionado por el sistema operativo (y no por el desarrollador), y ventanas de publicación controladas por tiendas de aplicaciones. La decisión entre desarrollo nativo (Kotlin/Swift) y multiplataforma (Flutter, React Native) no es una preferencia estética: tiene consecuencias medibles en rendimiento, acceso a APIs del dispositivo, tamaño del binario y tiempo de llegada al mercado, razón por la cual la rúbrica exige justificar la elección con al menos dos criterios cuantitativos.

3.4. Integración y entrega continuas (CI/CD)

La entrega continua se apoya en un *pipeline* automatizado que ejecuta pruebas, empaqueta artefactos y despliega en ambientes segregados sin intervención manual en cada paso. La premisa subyacente (cultura DevOps) es que ningún cambio debería llegar a producción sin pasar por los mismos controles automatizados que cualquier otro cambio; el pipeline actúa como *quality gate*: si la integración falla, el despliegue no ocurre.

3.5. Observabilidad distribuida

La observabilidad de un sistema distribuido se sustenta convencionalmente en tres señales complementarias: métricas (agregados numéricos con etiquetas, por ejemplo tasa de peticiones por ruta), registros o *logs* (eventos discretos, idealmente estructurados) y trazas (secuencias de *spans* correlacionados por un identificador común que permiten reconstruir el camino de una petición a través de varios servicios). El estándar OpenTelemetry busca unificar la instrumentación de estas tres señales de forma independiente del proveedor de observabilidad que las consuma después (Prometheus, Grafana, etc.).

3.6. Modelo de calidad ISO/IEC 25010:2023

El modelo ISO/IEC 25010:2023 organiza la calidad de un producto de software en ocho características — adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad,

fiabilidad, seguridad, mantenibilidad y portabilidad — cada una descompuesta en subcaracterísticas medibles mediante métricas objetivas. Una evaluación sería bajo este modelo no describe la calidad de forma cualitativa, sino que fija un umbral objetivo por métrica y reporta si el sistema lo alcanza, con el nivel de rigor de un experimento controlado (media, desviación típica e intervalo de confianza sobre repeticiones).

4. Arquitectura del sistema

4.1. Vistas C4 y evolución respecto a E3

Las vistas C4 de nivel 1 (contexto) y nivel 2 (contenedores) construidas en E3 — frontend con Spring Cloud Gateway enrutando hacia los seis microservicios de Productos, Inventario, Pedidos, Órdenes y proveedores, Usuarios y Ventas — se mantienen vigentes en su forma general: esta entrega no agrega ni elimina microservicios, refactoriza su interior y asegura su conexión a la base de datos distribuida. El cambio relevante a nivel de contenedor es que **todos** los microservicios usan ahora CockroachDB como base real (Sección 4.4), mientras que en E3 esto era un perfil experimental paralelo a la base compartida usada para la validación funcional. La Figura 1 consolida esta vista actualizada: capa de clientes (SPA web y app Android), capa de acceso y seguridad (Caddy y el Gateway con filtro JWT), los seis microservicios y **armado-ia** sobre la red interna de Docker, el clúster CockroachDB de tres nodos con replicación Raft, y los dos servicios externos (SMTP para OTP, Amazon Bedrock para las explicaciones de **armado-ia**).

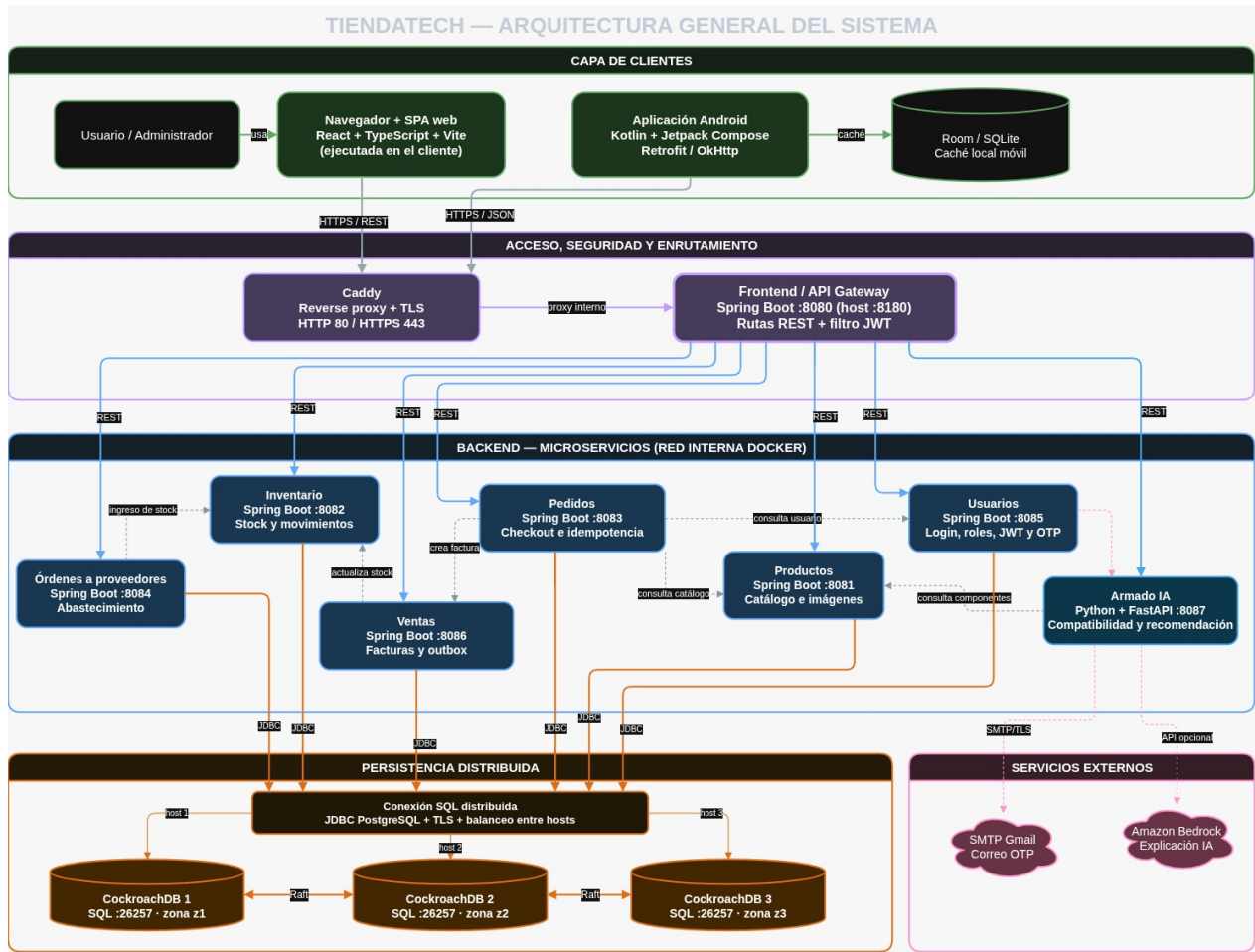


Figura 1: Arquitectura general de TiendaTech en la Entrega 4, con la persistencia distribuida ya unificada sobre CockroachDB.

4.2. Refactor en capas y principios SOLID (D1.1)

Los seis microservicios Java del backend (usuarios, productos-service, inventario-service, pedidos-service, ordenes-proveedores-service y ventas-service) quedaron organizados en los cuatro paquetes exigidos: **presentation** (controladores REST y DTO de entrada/salida), **application** (servicios de caso de uso, sin acceso directo a JDBC ni a HTTP saliente), **domain** (entidades y *puertos*, es decir, interfaces que declaran lo que el dominio necesita del exterior) e **infrastructure** (adaptadores concretos: repositorios JDBC, clientes HTTP, configuración de seguridad). El microservicio **armado-ia**, escrito en Python, mantiene una separación equivalente por responsabilidad (**clients/**, **explicacion/**, capa de servicio) sin adoptar literalmente la misma nomenclatura de paquetes Java.

El patrón de puerto y adaptador se aplica de forma consistente: **ordenes-proveedores-service** y **ventas-service** declaran **InventarioPort** en **domain**, implementado por

`InventarioClient` en `infrastructure`; `ventas-service` añade además `FacturaStore` y `FacturaOutboxStore` como puertos de persistencia; `pedidos-service` es el caso más extenso, con puertos dedicados para cada colaborador externo (`ProductoPort`, `UsuarioPort`, `FacturaPort`, `CrdbProbePort`, `CrdbRetryPort`, `CrdbMetricsPort`, `BusinessMetricsPort`); y `usuarios` generaliza la convención a `domain/port/out` (`TokenPort`, `UsuarioRepositoryPort`, `EmailPort`, `PasswordHasher`, `OtpStorePort`, entre otros), cada uno con su adaptador en `infrastructure/persistence/adapter` o `infrastructure/security`.

Esta organización materializa dos principios SOLID de forma verificable en el código. El **principio de responsabilidad única** (SRP) separa qué hace el sistema (`application`) de cómo lo hace (`infrastructure`): por ejemplo, `OrdenCompraService` orquesta la recepción de una orden de compra sin saber si el inventario se actualiza vía HTTP, cola de mensajes o llamada local. El **principio de inversión de dependencias** (DIP) se sostiene en que las clases de `application` dependen únicamente de las interfaces declaradas en `domain`, nunca de una clase concreta de `infrastructure`; Spring resuelve la implementación real (`InventarioClient`, `JdbcOrdenCompraRepository`, etc.) por inyección en tiempo de arranque. Esto permite, por ejemplo, sustituir `InventarioClient` por un doble de prueba sin tocar `OrdenCompraService`.

4.3. Patrones GoF aplicados (D1.2)

La documentación completa de esta decisión vive en `docs/adr/ADR-005-patrones-gof.md`. La auditoría registrada en ese ADR es deliberadamente conservadora: de los ocho servicios del backend, la mayoría implementa una arquitectura por capas limpia sin patrones GoF genuinos, y lo que a primera vista parece un patrón resulta ser inyección de dependencias de Spring, un *Repository* (Fowler, no GoF) con una sola implementación, o un *builder* de una librería de terceros. En vez de etiquetar falsamente esos casos, el equipo documenta cinco patrones GoF con evidencia real en el código:

1. **Strategy** (comportamiento): `ExplicacionClient` (`services/armado-ia/app/explicacion/client.py`) como interfaz, con dos estrategias intercambiables — `BedrockExplicacionClient` (usa AWS Bedrock) y `DeterministicExplicacionClient` (redacción sin LLM) — seleccionadas en runtime por `ExplicacionService`.
2. **Adapter** (estructural): `BedrockExplicacionClient` envuelve el SDK `boto3` y `ProductoClient` (`services/armado-ia/app/clients/producto_client.py`) envuelve `httpx`, ambos traduciendo su fuente externa a tipos de dominio propios.
3. **Decorator** (estructural): `TrustedUserHeaderRequest`, clase interna de `JwtGatewayFilter` (`Apps/web/frontend`), extiende `HttpServletRequestWrapper` para inyectar cabeceras de identidad verificadas sin mutar la petición original.
4. **Chain of Responsibility** (comportamiento): `ExplanationValidationHandler` y sus implementaciones concretas (`NonEmptyExplanationHandler`, `VerifiedHardwareToken`).

nHandler) forman una cadena de validadores anti-alucinación sobre la respuesta del LLM.

5. **Template Method** (comportamiento): el método `validate()` de `ExplanationValidationHandler` fija el algoritmo común (ejecutar regla, devolver problema si existe, delegar al siguiente) y delega el paso variable a `_validate_current()` en cada subclase.

El ADR también deja constancia explícita de los patrones evaluados y descartados por no tener estructura GoF real en este backend (Strategy sobre puertos con una sola implementación, Factory Method/Abstract Factory sobre `@Bean` sin jerarquía Creator, Builder de terceros, Observer, Facade, Command, State y Singleton), y documenta por separado — sin mezclarlos con el catálogo GoF — otros patrones legítimos presentes en el proyecto: *Repository* (Fowler) en los seis servicios Java, *Outbox* en `ventas-service` y *Circuit Breaker/Retry* (Nygard) en `pedidos-service`, `ordenes-proveedores-service` y `armado-ia`.

4.4. Persistencia distribuida operando (D4.2)

El clúster CockroachDB de tres nodos heredado de la Entrega 3 (puertos 26257/26258/26259) se mantiene operando y es el que sostiene la persistencia de los seis microservicios Java. Sobre esa base, esta entrega incorpora dos mejoras verificables en `docker-compose.yml`.

La primera es de **disponibilidad**: los siete servicios que dependen de una base de datos o exponen un endpoint HTTP (`pedidos`, `productos`, `inventario`, `ventas`, `ordenes-proveedores`, `usuarios` y `armado-ia`) ahora declaran un bloque `healthcheck`: contra `/actuador/health` (o el equivalente del servicio Python), con reintentos y periodo de arranque configurados; antes de esta entrega, `ventas-crdb-service` y `ordenes-proveedores-crdb-service` carecían de este chequeo, lo que impedía que `docker compose` reportara con precisión cuándo el sistema completo estaba realmente listo para recibir tráfico.

La segunda es de **seguridad de la conexión**: la cadena de conexión de los seis servicios Java se migró de `jdbc:postgresql://<IP fija de AWS>:26257,...&sslmode=disable` — una conexión sin cifrar contra una dirección IP fija — a `jdbc:postgresql://host.docker.internal:26257,...&sslmode=verify-ca&sslrootcert=/app/crdb-certs/ca.crt&sslcert=/app/crdb-certs/client.jhinson.crt&sslkey=/app/crdb-certs/client.jhinson.key.pk8`, es decir, autenticación mutua por certificado (mTLS) en vez de una conexión abierta. Los tres archivos de certificado se montan como volúmenes de solo lectura (`./Apps/web/infra/crdb-certs/*.crt|.key.pk8:...:ro`) en cada contenedor. Este cambio requirió reconciliar manualmente un conflicto de fusión en `docker-compose.yml` porque ambas mejoras (`healthchecks` y mTLS) se desarrollaron en paralelo sobre los mismos bloques de servicio; el resultado final conserva ambas.

4.5. Stack tecnológico

La Tabla 2 consolida las versiones vigentes verificadas directamente en los archivos de build del repositorio (`pom.xml`, `package.json`, `libs.versions.toml`), actualizando la Tabla de herramientas de `docs/entrega3/PFC3.tex` con lo efectivamente usado en esta entrega.

Cuadro 2: Stack tecnológico consolidado por capa (Entrega 4).

Capa	Tecnología	Versión verificada
Backend (6 microservicios Java)	Java / Spring Boot	Java 21, Spring Boot 3.3.5 (<code>pom.xml</code> de <code>pedidos-service</code>)
Backend (armado-ia)	Python	FastAPI, pytest + ruff en CI
Persistencia distribuida	CockroachDB	v23.2.4 verificado en la prueba de integración con Testcontainers de <code>pedidos-service</code> ; v23.1.13 en el nodo local de <code>docker-compose.override.yml</code>
Web (SPA)	React / TypeScript	React 18.3.1, <code>react-router-dom</code> 7.18.2, TypeScript 5.9.2, Vite 7.1.2 (<code>package.json</code>)
Web (Gateway)	Spring Cloud Gateway	Empaquetado junto al frontend, ver Dockerfile multi-stage (Sección 5.2)
Móvil	Kotlin / Jetpack Compose	Kotlin 2.2.10, Compose BOM 2026.02.01, Hilt 2.60.1, <code>compileSdk/targetSdk</code> 37, <code>minSdk</code> 26 (<code>libs.versions.toml</code> , <code>build.gradle.kts</code>)
Contenerización	Docker / Docker Compose	Build multi-stage por servicio; publicación multi-arquitectura <code>linux/amd64</code> , <code>linux/arm64</code> en <code>publish-images.yml</code>
CI/CD	GitHub Actions	<code>ci.yml</code> (3 jobs) + <code>publish-images.yml</code> , ver Sección 7.3

5. Aplicación web

5.1. Funcionalidad (D2.1)

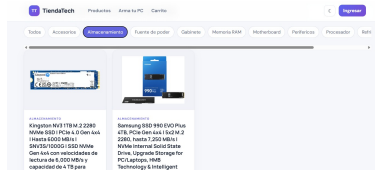
La SPA (`Apps/web/frontend/webapp`), construida con React 18 y `react-router-dom`, expone doce rutas declaradas en `App.tsx`: catálogo (`/`), detalle de producto (`/producto/:id`), armado de PC (`/armado`), carrito (`/carrito`), pago (`/pago`), factura (`/factura/:id?`), autenticación (`/login`, `/registro`, `/recuperacion`), cuenta (`/cuenta`), panel de trabajador (`/trabajador`) y administración (`/admin`), superando ampliamente el mínimo de cinco exigido por la rúbrica.

La sesión se identifica mediante un token JWT emitido por `usuarios-crdb-service`. El módulo `src/services/session.ts` guarda el token exclusivamente en `sessionStorage` (nunca en `localStorage`: al cargar, el propio módulo purga activamente cualquier clave de sesión que hubiera quedado en `localStorage` de una versión anterior), cumpliendo la restricción explícita de la rúbrica sobre el manejo del token. La ruta `/admin` está protegida por un componente `AdminRoute` que valida el rol del usuario (`id_rol === 1` o `rol === "admin"`) y redirige a `/login` si no se cumple. Adicionalmente, `App.tsx` implementa expiración de sesión por inactividad (30 minutos sin interacción del usuario cierran la sesión automáticamente).

La internacionalización declarada como deseable en la rúbrica (`react-i18next` con al menos español e inglés) no está implementada: toda la interfaz está codificada directamente en español. Se documenta aquí como brecha conocida en vez de omitirla.



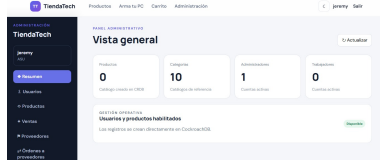
(a) Página principal (/)



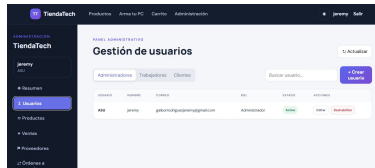
(b) Catálogo filtrado por categoría



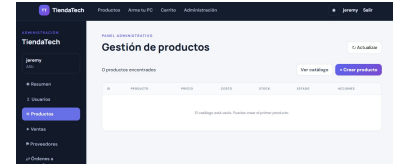
(c) Armado de PC (/armado)



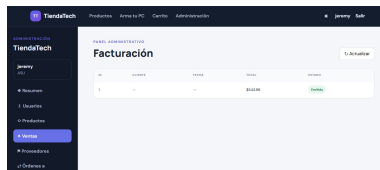
(d) Panel administrativo, resumen (/admin)



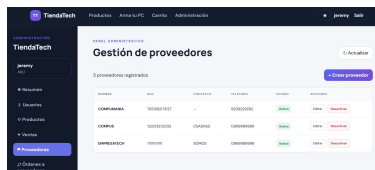
(e) Administración de usuarios



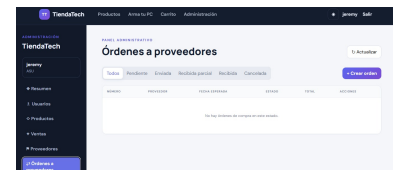
(f) Administración de productos



(g) Facturación



(h) Administración de proveedores



(i) Órdenes a proveedores

Figura 2: Capturas de las rutas principales de la SPA (página principal, catálogo, armado de PC) y del panel de administración (/admin), que cubre las seis entidades de negocio del sistema. La interfaz está codificada íntegramente en español, consistente con la ausencia de `react-i18next` señalada arriba.

5.2. Calidad (D2.2)

El `package.json` de la webapp no declara `eslint` ni `prettier` como dependencias de desarrollo, por lo que no hay una verificación de estilo o de reglas de código automatizada más allá de la que impone el propio compilador de TypeScript (`tsc -b`) durante el paso de `build`. Tampoco hay un framework de pruebas configurado (ni Vitest ni Jest aparecen en `devDependencies`), por lo que no existe cobertura de pruebas automatizada para la SPA en este momento; esta ausencia se retoma como brecha en la Sección 10.

El empaquetado sí está resuelto correctamente: `Apps/web/frontend/Dockerfile` usa una construcción *multi-stage* de tres etapas — `node:22-alpine` para compilar la SPA con `npm run build`, `maven:3.9-eclipse-temurin-17` para empaquetar el frontend Spring Cloud Gateway junto con los estáticos ya compilados de React, y `eclipse-temurin:17-jre` como imagen final de ejecución — lo que mantiene la imagen de producción libre de herramientas de compilación.

6. Aplicación móvil

6.1. Funcionalidad (D3.1)

`Apps/mobile` es una aplicación Android nativa en Kotlin con Jetpack Compose, organizada por *feature* (`auth`, `catalog`, `cart`, `orders`, `account`, `scanner`, `notifications`), cada una dividida internamente en `data/domain/ui`, con inyección de dependencias vía Hilt y comunicación HTTP con el backend mediante Retrofit y OkHttp. La capa `core/database` usa Room para cachear productos y categorías localmente (`ProductCacheDao`, `CategoryCacheDao`, `CacheMetadataDao`), permitiendo operar el catálogo con datos ya vistos aunque la conexión falle.

La aplicación utiliza dos capacidades del dispositivo, superando el mínimo exigido por la rúbrica. La primera es la **cámara**: la *feature scanner* usa `CameraX` (`androidx.camera.core.camera2/lifecycle/view`) junto con ML Kit Barcode Scanning para leer códigos de barras de productos y resolverlos contra el catálogo (`BarcodeAnalyzer`, `BarcodeLookup`). La segunda son las **notificaciones nativas**: `AndroidOrderNotificationPublisher` publica notificaciones reales del sistema operativo (`API NotificationManager/NotificationCompat`, con el permiso `POST_NOTIFICATIONS` declarado en el manifiesto) para informar cambios de estado en las órdenes del usuario.

Los flujos de negocio cubiertos son autenticación (`login/registro`, *feature/auth*), catálogo y detalle de producto (*feature/catalog*), carrito (*feature/cart*), órdenes y su historial (*feature/orders*), y cuenta con checkout (*feature/account*, incluye `CheckoutScreen`). La sesión se protege de forma más estricta que en la web: el token no se guarda en texto plano sino cifrado mediante Android Keystore (`AndroidKeystoreTokenCipher`, `SecureSessionStore`), y `AuthInterceptor` lo adjunta automáticamente a las peticiones salientes de Retrofit.

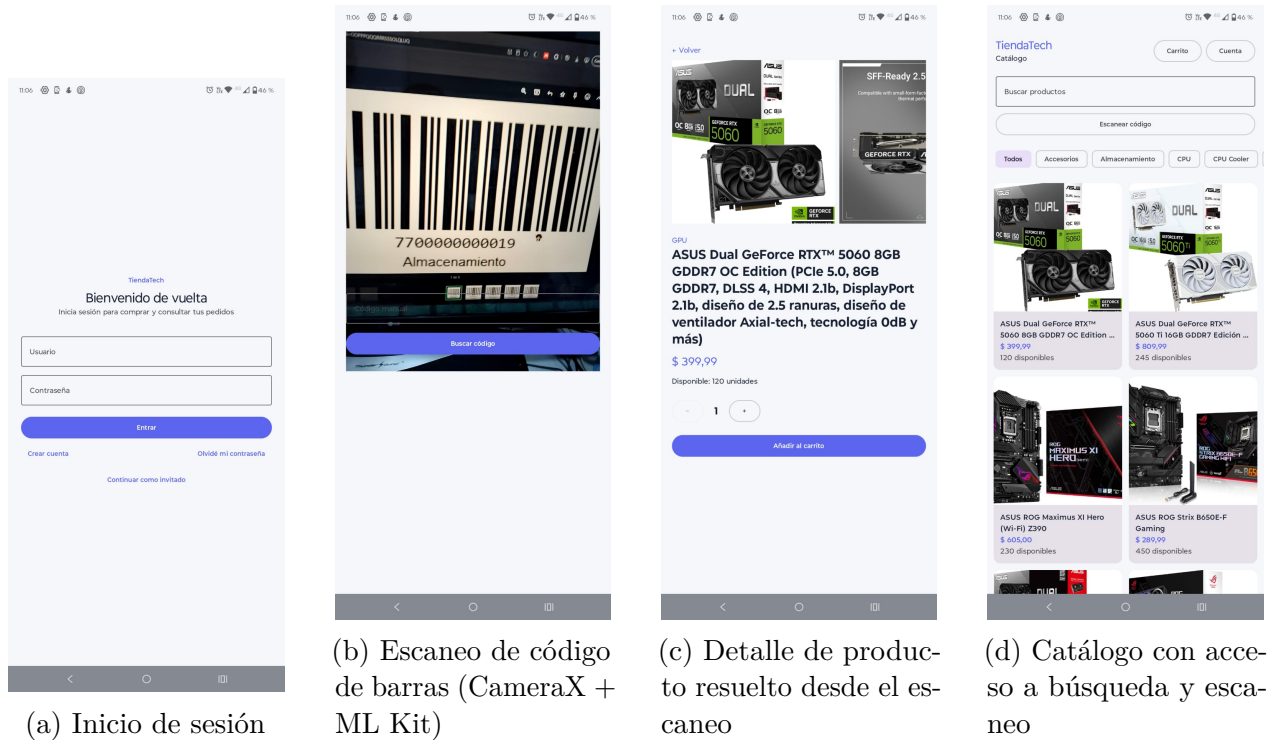


Figura 3: Flujo de autenticación y evidencia visual de la capacidad de cámara (escaneo de código de barras vía CameraX y ML Kit) en la aplicación móvil nativa. La segunda capacidad usada, notificaciones nativas (`AndroidOrderNotificationPublisher`), no se ilustra aquí para evitar exponer datos personales reales de la cuenta de prueba visibles en esa pantalla.

6.2. Calidad (D3.2)

La aplicación cuenta con dos niveles de prueba verificados en el repositorio. Existen pruebas instrumentadas (`app/src/androidTest`) que corren sobre un dispositivo o emulador real: `TiendaTechDatabaseTest` valida el esquema Room y `NotificationIntegrationTest` valida la publicación de notificaciones nativas, además de la plantilla `ExampleInstrumentedTest` generada por Android Studio. Adicionalmente — y a diferencia de lo que se había señalado como pendiente de confirmar en la auditoría interna previa a esta entrega — sí existen pruebas unitarias reales sobre `app/src/test`: doce archivos que cubren `ViewModels` (`ScannerViewModelTest`, usando `kotlinx.coroutines.test` con `StandardTestDispatcher` y `runTest` para probar corrutinas de forma determinista), repositorios (`AuthRepositoryTest`, `CartContractTest`, `OrdersContractTest`), validadores (`AuthValidatorTest`), mapeadores (`CatalogMapperTest`) y almacenamiento seguro de sesión (`SecureSessionStoreTest`), todas con JUnit 4 y `okhttp-mockwebserver` para simular el backend.

El job `android-mobile` de `.github/workflows/ci.yml` ejecuta `lintDebugtestDebugUnitTestassembleDebugassembleDebugAndroidTest` en cada push y publica el APK de depuración y el APK de pruebas instrumentadas como artefactos descargables del pipeline, cumpliendo la exigencia de que el CI produzca un artefacto instalable de la app móvil.

7. Pruebas y CI/CD

7.1. Pirámide de pruebas (D5.1)

La cobertura de pruebas del backend es desigual entre servicios y se documenta aquí con exactitud en vez de promediarla. Cuatro de los seis microservicios Java tienen pruebas unitarias reales con JUnit 5: `usuarios` (filtros JWT, guardia de acceso, sesiones), `productos-service` (controlador, manejador de excepciones, validación de DTO), `inventario-service` (idempotencia, DTO) y `pedidos-service`, que además de sus pruebas unitarias (`CarritoServiceTest`, `OrdenServiceTest`, `MetodoPagoServiceTest`) incluye la única prueba de **integración** real del backend: `CockroachDbIntegrationTest`, que levanta un contenedor `cockroachdb/cockroach:v23.2.4` real vía Testcontainers (`@Testcontainers`, `@Container`) en lugar de simular la base de datos.

Los otros dos microservicios Java, `ordenes-proveedores-service` y `ventas-service`, no tienen ninguna prueba (`src/test` no existe en ninguno de los dos). Esto es una brecha significativa precisamente porque ambos concentran la lógica de negocio más nueva y más sensible de esta entrega: la recepción parcial de órdenes de compra, el recálculo de costo por promedio ponderado y la auto-desactivación de productos cuando el costo supera el precio. El plan de cierre del equipo (`docs/plan-cierre-e4.md`) declaraba esta lógica como prioridad de cobertura por encima de perseguir un 70 % exacto en toda la superficie del código; a la fecha de esta sección, esa prioridad está declarada pero no ejecutada, y se deja constancia de ello en la Discusión (Sección 10) en vez de reportarla como resuelta.

No existe una carpeta `tests/` en la raíz del repositorio, por lo que tampoco hay pruebas end-to-end de la web con Playwright ni pruebas de carga con Locust. La aplicación móvil sí cierra su propia pirámide de forma más completa, con pruebas unitarias y pruebas instrumentadas (detalladas en la Sección 6.2).

7.2. Contratos de consumidor Pact (D4.1)

Como capa de *contrato* de la pirámide de pruebas, la rúbrica exige *consumer-driven contract testing* con Pact entre los clientes (web y móvil, como consumidores) y el backend (como proveedor). A la fecha de cierre de esta sección no existe en el repositorio ninguna carpeta `tests/contract` ni configuración de Pact (no hay dependencias de `pact-jvm` en ningún `pom.xml` del backend, ni de un *Pact broker* referenciado en el pipeline de CI/CD). Este criterio se declara explícitamente como **no cumplido** en el estado actual del proyecto. La razón no es desconocimiento de la herramienta, sino una decisión de priorización bajo el tiempo disponible del equipo: se optó por asegurar primero que la persistencia distribuida operara de forma segura (Sección 4.4) y que la calidad de las aplicaciones cliente avanzara (Secciones 5 y 6), antes de introducir una herramienta de verificación de contratos que ninguno de los servicios consumidores tenía previamente. Queda como trabajo inmediato siguiente, con una ruta ya decidida en `docs/plan-cierre-e4.md`: priorizar el contrato *web↔backend* (el cliente

con más tráfico y más rutas) antes que el contrato del consumidor móvil, de modo que exista al menos un contrato verificado en verde en lugar de dos a medio terminar.

7.3. Pipeline CI/CD end-to-end (D5.2)

El repositorio tiene dos workflows de GitHub Actions. `.github/workflows/ci.yml` corre en cada *push/pull request* y define tres jobs: `android-mobile` (lint, pruebas unitarias e instrumentadas, y publicación del APK como artefacto — ver Sección 6.2), `crdb-tests` (ejecuta `mvn verify` sobre `pedidos-service`, incluyendo la prueba de integración con Testcontainers) y `armado-ia-tests` (pruebas con `pytest` y lint con `ruff` sobre el microservicio Python). `.github/workflows/publish-images.yml`, agregado en esta entrega, construye y publica en un registro Docker las ocho imágenes del sistema (frontend y los siete microservicios) en una matriz paralela, con soporte multi-arquitectura (`linux/amd64`, `linux/arm64`) y cacheo de capas vía `type=gha`.

Comparado contra los siete jobs que exige la rúbrica (`lint`, `test-backend`, `test-web`, `test-mobile`, `build-images`, `build-mobile-apk`, `integration`), el pipeline actual cubre de forma parcial `test-mobile` y `build-mobile-apk` (ambos dentro de `android-mobile`), `integration` (dentro de `crdb-tests`, pero limitada a un solo servicio) y `build-images` (`publish-images.yml`). Quedan sin job dedicado: `lint` para los servicios Java y para la web (solo `armado-ia` tiene lint automatizado, vía `ruff`), y `test-backend`/`test-web` como jobs que corran las pruebas de *todos* los servicios Java y de la SPA respectivamente — esta última, además, no puede ejecutarse todavía porque, como se documentó en la Sección 5.2, la web no tiene ningún framework de pruebas configurado. Una diferencia adicional frente a lo esperado por la rúbrica es que `publish-images.yml` publica en Docker Hub (`docker/login-action` contra `vars.DOCKERHUB_USERNAME`) y no en GitHub Container Registry (GHCR) como se sugiere en la guía de la entrega; el resultado funcional — imágenes versionadas y publicadas automáticamente — es equivalente, pero el registro concreto difiere del nombrado explícitamente en la rúbrica.

8. Observabilidad

Este criterio (D6: instrumentación con OpenTelemetry, métricas Prometheus, registros estructurados con `trace_id` y dashboard operativo en Grafana con los seis paneles obligatorios) se declara **no implementado** en esta entrega. No hay carpeta `ops/` en el repositorio, ni `opentelemetry-spring-boot-starter`, ni `prometheus/grafana` en `docker-compose.yml`, ni logs en formato JSON con identificador de traza en la configuración de Logback de los servicios revisados. Esta decisión de alcance ya fue declarada en la Sección 1.1 y se retoma con su justificación completa en la Discusión (Sección 10): la instrumentación de las tres señales de observabilidad es, de todo lo pendiente en la rúbrica, la de mayor costo de infraestructura por punto de nota, y compite directamente por el mismo tiempo que se usó para asegurar la persistencia distribuida seleccionada como prioridad (Sección 4.4).

9. Evaluación ISO/IEC 25010

Este criterio (D7: medir cinco características del modelo ISO/IEC 25010:2023 con métricas objetivas, repeticiones y un intervalo de confianza del 95 %) se declara **no implementado** en esta entrega, como consecuencia directa de la Sección 8: las métricas de fiabilidad (tasa de errores 5xx) y de eficiencia de desempeño (latencia p95 bajo carga con Locust) que exige el protocolo experimental de la rúbrica dependen de la instrumentación de observabilidad, que no se implementó. No existe `docs/experimentos/protocolo-e4.md` ni `docs/experimentos/resultados/iso25010.csv` en el repositorio. El equipo prefiere declarar esta ausencia con claridad antes que reportar métricas medidas de forma artesanal o no reproducible, lo cual violaría el mismo estándar de rigor experimental que la rúbrica exige.

10. Discusión y amenazas a la validez

10.1. Interpretación de resultados y alcance no cubierto

La Entrega 3 cerró declarando como trabajo posterior explícito la calidad de código, la observabilidad y la evaluación experimental formal del sistema (Sección 1). Frente a esa línea base, esta entrega avanza de forma desigual entre esos tres frentes, y esa desigualdad es en sí misma el hallazgo principal de la Discusión.

El avance más sólido es arquitectónico: los seis microservicios Java completaron el refactor en capas con puertos y adaptadores (Sección 4), y la persistencia distribuida pasó de ser un perfil experimental paralelo en E3 a ser la base de datos real del sistema, ahora protegida con mTLS. Este resultado es consistente con la prioridad que el equipo declaró en `docs/plan-ci-erre-e4.md`: asegurar primero la infraestructura de la que todo lo demás depende.

El avance en calidad de código es parcial y concentrado: cuatro de los seis microservicios Java tienen pruebas unitarias reales, mientras que los dos que concentran la lógica de negocio más nueva de esta entrega (recepción de órdenes, costeo, auto-desactivación) no tienen ninguna (Sección 7.1). Esto invierte la prioridad que el propio equipo se había fijado por escrito — cubrir primero la lógica nueva antes que perseguir cobertura pareja — y es una inconsistencia entre lo planeado y lo ejecutado que este documento prefiere señalar explícitamente en vez de omitir. De forma similar, el pipeline de CI/CD cubre de forma parcial cuatro de los siete *jobs* esperados (Sección 7.3), y los contratos Pact no se iniciaron (Sección 7.2).

La evaluación experimental formal — el tercer frente declarado pendiente en E3 — sigue sin abordarse en E4: la observabilidad distribuida y la evaluación ISO/IEC 25010 (Secciones 8 y 9) se declaran explícitamente fuera de alcance, no por desconocimiento de las herramientas sino porque instrumentar las tres señales de observabilidad (métricas, registros, trazas) y luego ejecutar un protocolo experimental con repeticiones e intervalos de confianza sobre esas métricas es, de todo lo pendiente en la rúbrica, lo que exige más tiempo de infraestructura por punto de nota disponible. Priorizar la persistencia distribuida seleccionada como base de

todo el sistema, en lugar de la observabilidad, fue una decisión consciente del equipo bajo la restricción de tiempo real que enfrentó esta entrega, y no un descuido.

La instrumentación de observabilidad distribuida (OpenTelemetry, Prometheus, Grafana) y la evaluación experimental ISO/IEC 25010 — que depende de tener esas métricas funcionando — quedaron fuera del alcance de esta entrega por restricción de tiempo del equipo, y se declaran explícitamente como trabajo futuro en las Conclusiones (Sección 11). No se fabrican métricas ni dashboards simulados para cubrir estos puntos (Secciones 8 y 9).

10.2. Amenazas a la validez (D9.3)

Sobre las mediciones y verificaciones reportadas en la Sección 7, corresponde declarar tres amenazas a su validez siguiendo la clasificación estándar de estudios empíricos en ingeniería de software.

Validez de constructo: la cobertura de pruebas se reportó por *presencia de archivos de prueba por servicio*, no por un porcentaje de líneas o ramas cubiertas medido con una herramienta como JaCoCo. Solo `pedidos-service` tiene JaCoCo configurado en su `pom.xml`; en el resto del backend, "tiene pruebas" y "tiene buena cobertura" no son estrictamente lo mismo, y este documento no confunde ambas cosas al reportar el estado del servicio.

Validez interna: la única prueba de integración real contra una base de datos (`CockroachDbIntegrationTest` en `pedidos-service`) corre contra un contenedor CockroachDB de un solo nodo levantado por Testcontainers, mientras que el sistema en ejecución real usa un clúster de tres nodos con mTLS. El comportamiento verificado en la prueba automatizada no cubre escenarios de consenso Raft entre réplicas, conflictos de rango entre nodos, ni la ruta de conexión cifrada.

Validez externa: las pruebas manuales de extremo a extremo de esta entrega (recepción de órdenes, actualización de inventario, auto-desactivación de productos) se ejecutaron principalmente contra el entorno local descrito en la Sección 10.4, con datos de catálogo sintéticos y volumen bajo. No hay evidencia todavía de que el mismo flujo se comporte igual bajo carga concurrente o con el volumen de datos del clúster de AWS, precisamente el tipo de medición que correspondería a una evaluación ISO/IEC 25010 (Sección 9).

10.3. Reflexión ética (D9.2)

Se toma como marco de referencia el *ACM Code of Ethics and Professional Conduct* (2018), aplicado a tres decisiones de diseño concretas de **TiendaTech**, no como una discusión abstracta.

La primera es la protección de credenciales y datos personales (principio 1.6, *Respect privacy*; principio 2.9, *Design for security*). Las contraseñas se almacenan con `BCryptPasswordEncoder` de factor 12, nunca en texto plano; el token de sesión se guarda cifrado con Android

Keystore en el cliente móvil y solo en `sessionStorage` (no en `localStorage` persistente) en el cliente web; y la conexión hacia la base de datos distribuida se cifra con autenticación mutua por certificado (mTLS) desde esta entrega, en lugar de la conexión sin cifrar heredada de E3.

La segunda es la decisión de auto-desactivar un producto cuando su costo recalculado supera su precio de venta (documentada en la Sección 4 y en el código de `inventario-service/productos-service`). Esta es una decisión automatizada con impacto económico directo sobre el catálogo visible al cliente, y por tanto queda dentro del alcance del principio 2.5 (*Give comprehensive evaluation of computer systems... including analysis of possible risks*) y del principio 1.2 (*Avoid harm*): vender por debajo del costo perjudicaría al negocio, pero desactivar un producto sin aviso también tiene un costo (pérdida de visibilidad, posible error de cálculo transitorio). Por eso la implementación no es una decisión silenciosa e irreversible: queda registrada en el log del servicio (`LOG.warn` con el producto, el costo recalculado y el precio vigente), es visible y reversible por un administrador humano, y la reactivación exige explícitamente que el precio ya no esté por debajo del costo antes de permitirla — es decir, el sistema no permite reactivar "a ciegas" un producto que seguiría vendiéndose con pérdida.

La tercera es la honestidad en la comunicación de resultados, que el Code of Ethics exige en el principio 2.6 (*Perform work only in areas of competence*) y, sobre todo, en el compromiso general de integridad profesional: este mismo documento declara explícitamente, sección por sección, qué partes de la rúbrica se cumplen, cuáles se cumplen de forma parcial y cuáles no se abordaron (Observabilidad e ISO/IEC 25010 en las Secciones 8 y 9; ausencia de pruebas en dos microservicios en la Sección 7; ausencia de contratos Pact en la Sección 7.2), en vez de presentar el proyecto como completo cuando no lo está.

10.4. Reproducibilidad (D9.4)

El sistema completo puede levantarse con `dockercomposeup--build` contra el clúster CockroachDB de AWS usando las credenciales por defecto embebidas en `docker-compose.yml`, pero esto ata cualquier prueba local a un recurso compartido por todo el equipo y a una base de datos que no se puede resetear libremente. Para resolver esto, `docker-compose.override.yml` — que `docker compose` carga automáticamente junto al archivo principal, sin modificarlo — añade un nodo CockroachDB local de un solo contenedor (`crdb-local`, modo `-insecure`, sin necesidad de certificados) y un segundo contenedor de inicialización (`crdb-local-init`) que carga, en orden, el esquema (`schema.sql`), los datos semilla (`seeds.sql`) y dos catálogos de referencia críticos para que el sistema funcione de punta a punta: `product-management-reference.sql` (categorías, marcas, IVA) y `inventario-movimientos-reference.sql` (tipos y subtipos de movimiento de inventario, cuya ausencia fue precisamente la causa raíz de un defecto de esta misma entrega, donde las recepciones de órdenes de compra no actualizaban el stock ni en el entorno local ni en AWS).

Esta reproducibilidad tiene un límite honesto que se documenta aquí en vez de omitirse: los servicios Java no apuntan automáticamente a `crdb-local` por defecto, sino al clús-

ter mTLS de AWS (Sección 4.4); para usar el entorno local hay que sobrescribir la variable de entorno `CRDB_DATASOURCE_URL` antes de levantar los contenedores (por ejemplo, `$env:CRDB_DATASOURCE_URL` en PowerShell), y esa variable es de alcance de sesión de terminal — si se abre una pestaña nueva sin volver a definirla, los contenedores reconstruidos en esa pestaña vuelven a apuntar por defecto a AWS mientras el resto del stack sigue apuntando a `crdb-local`, produciendo un entorno mixto (*split-brain*) difícil de diagnosticar. El equipo confirmó este comportamiento durante las pruebas de esta entrega. La reproducibilidad "desde cero" en una máquina limpia es alcanzable pero no todavía a prueba de error humano; formalizar la variable en un archivo `.env` versionado (con placeholder) en vez de exportarla manualmente por sesión queda como mejora concreta pendiente.

11. Conclusiones y trabajo futuro

11.1. Cierre del ciclo E1–E4

La Tabla 3 resume el aporte específico de cada entrega, construida a partir de la lectura directa de `docs/entrega1/entrega1.pdf`, `docs/entrega2/entrega2.pdf`, `docs/entrega3/PFC3.tex` y la evidencia técnica en `docs/evidencias/`.

Cuadro 3: Trazabilidad de la evolución del proyecto entre E1 y E4.

Entrega	Enfoque	Aporte específico
E1 (27 may. 2026)	Propuesta y análisis	Especificación del problema (brecha de requerimientos formales de compatibilidad de hardware), delimitación explícita de la contribución al núcleo transaccional (autenticación, carrito, motor de compatibilidad, interfaz base del asistente inteligente) y roles del equipo (Arquitecto, Líder de Desarrollo, Responsable de Calidad, Responsable de Documentación).
E2 (6 jun. 2026)	Diseño arquitectónico	Diagramas C4 (contexto, contenedores, componentes), definición inicial de los servicios REST por dominio, contrato OpenAPI 3.0 y estrategia de contenerización con Docker para la arquitectura de microservicios propuesta.
E3 (28 jul. 2026)	Implementación y validación	Despliegue contenerizado real del frontend y seis microservicios sobre una base compartida, validado con 10 casos funcionales y 210 solicitudes (100 % de éxito, p95 <52ms). En paralelo, un perfil experimental desplegó un clúster CockroachDB de 3 nodos con fragmentación de <code>pedidos.orden</code> por trimestre (ADR-003) y factor de replicación 3 (ADR-004), verificado con pruebas de tolerancia a fallos documentadas en video (<code>docs/evidencias/tolerancia_fallos.mp4</code>), además de un pipeline analítico PySpark sobre 600 000 órdenes.
E4 (esta entrega)	Calidad, persistencia distribuida en producción y cierre	CockroachDB deja de ser un perfil experimental y pasa a ser la base de datos real de los seis microservicios en <code>docker-compose.yml</code> , ahora con conexión cifrada por certificado (mTLS); los seis servicios Java se refactorizaron a la arquitectura de puertos y adaptadores en cuatro capas, con 5 patrones GoF documentados (ADR-005); se corrigió el almacenamiento del token JWT en la web; se amplió — de forma desigual, ver Sección 7 — la cobertura de pruebas y el pipeline de CI/CD con publicación de imágenes Docker; y se declaran explícitamente fuera de alcance la observabilidad distribuida, la evaluación ISO/IEC 25010 y los contratos Pact.

Dos piezas de evidencia en `docs/evidencias/` documentan específicamente la transición entre el perfil experimental de E3 y el uso productivo de CockroachDB en E4: `integracion-pedidos-crdb.md` y `colision-serializable-controlada.md`, esta última relevante para el aislamiento `SERIALIZABLE` aplicado en `ordenes-proveedores-service` al enviar y cancelar órdenes de compra (Sección 4).

11.2. Conclusiones

Contrastado contra el alcance declarado en la Sección 1.1, el equipo cumplió de forma completa el refactor en capas y los patrones GoF (D1), avanzó de forma sólida en la persistencia distribuida segura (D4.2) y en la funcionalidad de ambas aplicaciones cliente (D2.1, D3.1), y

corrigió el manejo inseguro del token JWT identificado como pendiente antes de esta entrega. Cumplió de forma parcial la calidad de las aplicaciones cliente (D2.2, D3.2) y la pirámide de pruebas del backend (D5.1), con una brecha concreta y ya identificada: los dos microservicios con la lógica de negocio más nueva de la entrega no tienen pruebas propias. No abordó, y lo declara así en vez de simularlo, los contratos de consumidor Pact (D4.1), la observabilidad distribuida (D6) y la evaluación experimental ISO/IEC 25010 (D7).

El resultado neto es un sistema que evolucionó genuinamente en la dirección que E3 dejó pendiente — de *funciona* a *se puede sostener y verificar* — pero que no alcanzó a cubrir la totalidad de esa transición en el tiempo disponible. Esta entrega documenta esa transición de forma honesta antes que completa: cada sección declara explícitamente su estado real, sin inflar lo logrado ni ocultar lo pendiente, siguiendo el mismo estándar de honestidad de alcance aplicado en las entregas anteriores del proyecto.

11.3. Trabajo futuro

Se declara explícitamente como trabajo pendiente, en orden de prioridad según `docs/plan-cierre-e4.md`: cerrar el contrato Pact web↔backend (Sección 7.2); escribir pruebas unitarias para `ordenes-proveedores-service` y `ventas-service` sobre la lógica de recepción, costeo y auto-desactivación (Sección 7.1); instrumentar observabilidad distribuida y, a partir de ella, ejecutar la evaluación ISO/IEC 25010 (Secciones 8 y 9); ampliar el pipeline de CI/CD a los siete jobs completos y migrar la publicación de imágenes de Docker Hub a GHCR (Sección 7.3); implementar internacionalización en la web (Sección 5.1); y formalizar la variable `CRDB_DATASOURCE_URL` del entorno local reproducible en un archivo `.env` versionado (Sección 10.4).

Referencias

- [1] R. Taft, I. Sharif, A. Matei et al., «CockroachDB: The resilient geo-distributed SQL database,» en *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, Portland, OR, EE. UU., 2020, págs. 1493-1509. DOI: [10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134).
- [2] M. T. Özsu y P. Valduriez, *Principles of Distributed Database Systems*, 4.^a ed. Cham, Suiza: Springer, 2020. DOI: [10.1007/978-3-030-26253-2](https://doi.org/10.1007/978-3-030-26253-2).
- [3] G. Márquez, F. Osses y H. Astudillo, «Review of Architectural Patterns and Tactics for Microservices in Academic and Industrial Literature,» *IEEE Latin America Transactions*, vol. 16, n.º 9, págs. 2321-2327, 2018. DOI: [10.1109/TLA.2018.8789551](https://doi.org/10.1109/TLA.2018.8789551).
- [4] C. Zhong, S. Li, H. Huang et al., «Domain-Driven Design for Microservices: An Evidence-Based Investigation,» *IEEE Transactions on Software Engineering*, vol. 50, n.º 6, págs. 1425-1447, 2024. DOI: [10.1109/TSE.2024.3385835](https://doi.org/10.1109/TSE.2024.3385835).
- [5] P. Nawrocki, K. Wrona, M. Marczak y B. Sniezynski, «A Comparison of Native and Cross-Platform Frameworks for Mobile Applications,» *Computer*, vol. 54, n.º 3, págs. 18-27, 2021. DOI: [10.1109/MC.2020.2983893](https://doi.org/10.1109/MC.2020.2983893).
- [6] M. Shahin, M. A. Babar y L. Zhu, «Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices,» *IEEE Access*, vol. 5, págs. 3909-3943, 2017. DOI: [10.1109/ACCESS.2017.2685629](https://doi.org/10.1109/ACCESS.2017.2685629).
- [7] M. Hui, L. Wang, H. Li et al., «Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,» *Journal of Systems and Software*, vol. 220, pág. 112 232, 2025. DOI: [10.1016/j.jss.2024.112232](https://doi.org/10.1016/j.jss.2024.112232).